

实验八 神经网络

（一）目的和要求

- （1）掌握使用 Keras 构建顺序神经网络模型的流程及相关函数的使用方法
- （2）会配置训练神经网络过程中的各种参数

（二）实验知识准备

Keras 提供了两种模型用于构建神经网络，分别是顺序模型和函数式 API 模型。本次实验以顺序模型为例。

顺序模型，也称为 Sequential 模型，是 Keras 中最常用的模型，其中预置了全连接层、卷积层、池化层、长短期记忆层和 GRU 层等多种神经层（将在后续项目中陆续介绍），使用这些预置的神经层可以快速构建神经网络模型。

顺序模型要求只能有一组输入张量和一组输出张量。各层之间按照先后顺序进行堆叠，前面一层的输出是后面一层的输入，通过不同层堆叠，构建神经网络。

（1）创建顺序网络模型

1. 顺序网络模型的结构

顺序全连接神经网络模型由输入层、隐藏层和输出层组成。

A. 输入层。输入层的神经元接收数据，并输入到神经网络中。在 Keras 中使用 Input() 函数构建输入层，格式如下。

```
tf.keras.Input(shape=None, batch_size=None, name=None, dtype=None)
```

其中，shape 表示输入数据的形状元组，默认为 None；batch_size 表示批量大小，默认为 None；name 表示当前层的名称，默认为 None；dtype 表示输入数据期望的数据类型，默认为 None。

例如，若网络模型的输入层有 4 个特征值，则构建输入层的代码如下。

```
x=tf.keras.Input(shape=(4,))
```

多数情况下，Keras 的 Sequential 模型可以不单独构建输入层，而是在新增第一个 Dense 隐藏层时，利用 input_shape 参数指定输入数据的形状。

在全连接网络中，当输入的数据不是一维数据，可以使用 Flatten() 函数将数据平展为一维数组，格式如下。

```
tf.keras.layers.Flatten(input_shape=None)
```

其中，`input_shape` 表示输入层尺寸的元组，如在 MNIST 数据集中，数据是 28×28 的二维结构，那么，将数据平展为一维张量的代码如下。

```
tf.keras.layers.Flatten(input_shape=(28,28))
```

B.隐藏层和输出层。在深度学习中，隐藏层主要包括全连接层、卷积层和池化层等。神经网络中隐藏层和输出层是具有计算功能的神经元，可以使用 `tf.keras.layers.Dense()` 函数构建全连接层，格式如下。

```
tf.keras.layers.Dense(units,activation=None,kernel_regularizer=None)
```

其中，`units` 表示神经元的个数；`activation` 表示激活函数，以字符串的形式给出，可以是 'relu'、'softmax'、'sigmoid'、'tanh' 等；`kernel_regularizer` 表示应用于权重的正则化函数，如 `tf.keras.regularizers.l1()` 或 `tf.keras.regularizers.l2()`。

例如，创建一个全连接层，有 8 个神经元，使用 ReLU 函数作为激活函数，代码如下。

```
tf.keras.layers.Dense(8, activation='relu')
```

2. 顺序网络模型的创建

创建一个顺序网络模型，格式如下。

```
tf.keras.models.Sequential(layer=None,name=None)
```

其中，`layer` 表示要添加到网络模型的层列表或者元组；`name` 表示网络模型的名称。

使用 `summary()` 函数，可以显示模型各层的参数信息，格式如下。

```
model.summary()
```

`model` 为已创建好的顺序模型。

(2) 编译顺序网络模型

创建好顺序网络模型后，须使用 `compile()` 函数进行编译，格式如下。

```
model.compile(loss=None,optimizer,metrics=None)
```

其中，`loss` 表示损失函数，默认为 `None`；`optimizer` 表示优化器对象，默认为 `None`；`metrics` 表示性能评估函数，用于评估模型的性能，默认为 `None`。

编译顺序网络模型需要设置损失函数、优化器和性能评估函数。

1. 损失函数

损失函数用于度量模型预测值与真实值的差异，是评估模型效果的一个重要

指标。损失函数值越小，表明模型的效果越好。Keras 中常用的损失函数如表 1 所示。

表 1 Keras 中常用的损失函数

损失函数字符串	损失函数形式	说 明
'mse'	tf.keras.losses.MSE()/ tf.keras.losses. MeanSquaredError()	计算预测值与真实值之差的平方和的均值，用于回归问题
'categorical_crossentropy'	tf.keras.losses. CategoricalCrossentropy()	计算标签值与预测值的交叉熵损失，用于多分类问题，要求标签为独热编码形式
'sparse_categorical_crossentropy'	tf.keras.losses. SparseCategoricalCrossentropy()	计算标签值与预测值的交叉熵损失，用于多分类问题，要求标签为序号编码形式
'binary_crossentropy'	tf.keras.losses.BinaryCrossentropy()	计算标签值与预测值的交叉熵损失，用于二分类问题

2.优化器

梯度下降算法，存在不足。首先，选择合适的学习率并不容易，过大的学习率会导致在训练过程中很难找到模型的最优参数，过小的学习率会导致模型训练很慢，需要较长的训练时间；其次，每次更新模型参数值，都只考虑当前计算出的梯度值，而直接忽略前面计算并使用过的梯度值。

针对这些不足，研究者提出了多种梯度下降优化算法。在这些算法中，如果学习率的值在模型开始训练时设置得较大，在模型训练过程中，按照一定的衰减率逐渐减小学习率的值，在加速模型训练的同时，在训练的后期更容易找到最优参数。

梯度下降优化算法还使用了动量来加速模型的训练。动量的工作原理与物理学中的惯性类似。动量的实现方式一般是在每次计算出当前的梯度值后，对当前梯度值和之前模型计算的梯度值取平均值，然后使用这个平均梯度值完成本次模型参数的更新。这样，之前计算出的梯度值可起到一个“惯性”的作用，从而影响本次模型参数更新的梯度值。

在 Keras 中，实现了多种梯度下降优化算法，简称优化器，如表 2 所示。

表 2 Keras 中常用的优化器

优化器字符串	优化器函数形式	说 明
'sgd'	tf.keras.optimizers.SGD (learning_rate=0.01, momentum=0.0)	随机梯度下降优化器

'adagrad'	tf.keras.optimizers.Adagrad (learning_rate=0.001)	能够在训练中自动对学习率进行调整，对于出现频率较低的参数采用较大的学习率进行更新；相反，对于出现频率较高的参数采用较小的学习率进行更新
'rmsprop'	tf.keras.optimizers.RMSprop(learning_rate=0.001, rho=0.9, momentum=0.0)	是 Adagrad 优化器的改进，其目的是在非凸背景下使效果更好，使用了简单动量
'adam'	tf.keras.optimizers.Adam(learning_rate=0.001, beta_1=0.9, beta_2=0.999)	结合了 Adagrad 和 RMSprop 优化器的优点，具有计算效率高、内存需求小等特点

3.性能评估函数

训练网络模型时，需要观察损失和分类精度等评估指标的变化，以便调整模型，取得更好的效果。性能评估函数既可以是 Keras 预定义的性能评估函数，也可以自定义性能评估函数。Keras 中常用的性能评估函数如表 3 所示。

表 3 Keras 中常用的性能评估函数

性能评估字符串	性能评估函数形式	说 明
'accuracy'	tf.keras.metrics.Accuracy()	准确率，计算预测值与标签值相等的概率
'binary_accuracy'	tf.keras.metrics.BinaryAccuracy (threshold=0.5)	计算预测值与标签值匹配的概率，用于二分类问题
'categorical_accuracy'	tf.keras.metrics. CategoricalAccuracy()	计算预测值与标签值匹配的概率，标签值和预测值均为独热编码形式，用于多分类问题
'sparse_categorical_accuracy'	tf.keras.metrics. SparseCategoricalAccuracy()	计算预测值与标签值匹配的概率，标签值为数值形式，预测值为独热编码形式，用于多分类问题
'binary_crossentropy'	tf.keras.metrics. BinaryCrossentropy()	计算标签值和预测值的交叉熵度量

在 compile() 函数中，性能评估函数须用方括号括起来，且可以同时列出多个性能评估函数，多个性能评估函数之间要用逗号分隔。

(3) 训练顺序网络模型

fit() 函数是训练网络模型常用的函数。在训练过程中，需要指定迭代轮数和每次批量大小，格式如下。

```
fit(x,y,batch_size=32,epochs=1,verbose=1,validation_split=
0.0,validation_data=None,shuffle=True)
```

其中，x 表示训练集的输入特征值，可以是数组、列表或字典；y 表示训练集的标签值，可以是数组或列表；batch_size 表示将数据集分批次处理，每一批

数据中包含的样本数量，默认为 32；`epochs` 表示对整体数据集训练的轮数，一轮是指完整数据集进行一次前向传播和一次反向传播，默认为 1；`verbose` 表示日志显示模式，其值为 0、1、2，0 为不显示任何内容，1 为显示进度条，2 为每轮迭代显示一行，默认为 1；`validation_split` 表示用作验证集的训练数据比例，默认为 0.0；`validation_data` 表示用于性能评估的数据，默认为 `None`；`shuffle` 表示每轮训练之前是否打乱数据，默认为 `True`。

`fit()`函数返回一个 `History` 对象，`History` 对象的 `history` 属性是一个字典，记录了训练时的训练损失函数值和评估指标。如果在训练过程中使用了验证集，那么相应的指标也会被记录下来，可以借助 `Matplotlib` 库绘制损失函数值和评估指标的变化曲线图。

（4）评估顺序网络模型

在训练网络模型结束后，可以使用 `evaluate()`函数对网络模型进行评估，格式如下。

```
evaluate(x,y,batch_size=None,verbose=1)
```

其中，`x` 表示用于评估网络模型的数据集的特征值；`y` 表示用于评估网络模型的数据集的标签值；`batch_size` 表示批量大小；`verbose` 表示日志显示模式，默认为 1。

例如，若网络模型 `model` 已训练完成，可以使用测试数据集评估该网络模型的性能，代码如下。

```
model.evaluate(x_test,y_test,batch_size=32)
```

（5）应用顺序网络模型

如果对训练好的网络模型的性能满意，就可以使用 `predict()`函数应用网络模型进行预测，格式如下。

```
predict (x,batch_size=None,verbose=0)
```

其中，`x` 表示数据集的特征值；`batch_size` 表示批量大小，默认为 `None`；`verbose` 表示日志显示模式，默认为 0。

`predict()`函数返回值为预测结果的 `NumPy` 数组。

（三）实验内容及步骤

（1）使用神经网络识别手写数字

手写数字数据集（MNIST）中的样本为手写数字图片。包含了 70000 张 0 至 9 的手写单张图片，其中，训练集 60000 张，测试集 10000 张。每张图片的标签值为图片中对应的数字，即 0 至 9，共 10 种。MNIST 数据集中每张图片的长和宽都为 28 像素，且均为 256 阶灰度图，即每个像素值在 0 至 255 之间。

1. 步骤一，导入本项目所需要的模块和包，从 Keras 中导入 MNIST 数据集，将训练集的特征值和标签值分别存储在 `x_train` 和 `y_train` 中，将测试集的特征值和标签值分别存储在 `x_test` 和 `y_test` 中。对特征值 `x_train` 和 `x_test` 进行标准化处理，使其取值范围为 0~1。如图 1 所示。

```
1 import tensorflow as tf
2 import numpy as np
3 import matplotlib.pyplot as plt
4 #导入MNIST数据集，将训练集和测试集存储在相应变量中
5 mnist = tf.keras.datasets.mnist
6 (x_train, y_train), (x_test, y_test) = mnist.load_data()
7 #特征值标准化处理
8 x_train, x_test = x_train / 255.0, x_test / 255.0
```

图 1 准备数据集

2. 步骤二，构建空的顺序网络模型，如图 2 所示。

```
1 #构建顺序网络模型
2 model = tf.keras.models.Sequential()           #构建空的顺序模型
3 #使用Flatten()函数将数据平展为一维数组
4 model.add(tf.keras.layers.Flatten(input_shape=(28, 28)))
5 #为网络模型添加隐藏层和输出层
6 model.add(tf.keras.layers.Dense(128, activation='relu'))
7 model.add(tf.keras.layers.Dense(10, activation='softmax'))
8 model.summary()
```

图 2 从构建网络模型

MNIST 数据集的图片是 28×28 的二维结构，须使用 `Flatten()` 函数将数据平展为一维数组。为网络模型添加隐藏层，节点个数为 128 个，使用 RELU 函数作为激活函数。为网络模型添加输出层，节点个数为 10，使用 SOFTMAX 函数作

为激活函数，并显示网络各层参数信息。

3. 步骤三，编译、训练和评估顺序网络模型，主要代码如图 3 所示。

```
1 #编译网络模型
2 model.compile(optimizer='adam',
3               loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=False),
4               metrics=['sparse_categorical_accuracy'])
5 #训练网络模型
6 model.fit(x_train,y_train,batch_size=32,epochs=5)
7 #评估网络模型
8 model.evaluate(x_test,y_test,batch_size=32,verbose=2)
```

图 3 编译、训练和评估顺序网络模型

编译顺序网络模型，其中，损失函数使用稀疏交叉熵损失函数，优化器使用 Adam 优化器，性能评估函数使用稀疏交叉熵准确率函数。训练顺序网络模型，其中，设置批量大小为 32，迭代次数为 5。使用测试集对顺序网络模型进行评估，其中，设置批量大小为 32，日志显示模式为 2。

4. 步骤四，应用顺序网络模型，示例代码如图 4 所示。

```
1 #应用顺序网络模型
2 for i in range(5):
3     t=np.random.randint(1,10000)    #生成随机整数t
4     x=tf.reshape(x_test[t],(1,28,28))
5     #应用网络模型
6     y_pred=np.argmax(model.predict(x),axis=1)
7     plt.subplot(1,5,i+1)             #创建子图
8     plt.rcParams['font.sans-serif']=['SimHei']
9     plt.axis("off")                  #设置不显示坐标轴
10    plt.imshow(x_test[t],cmap='gray')
11    title="标签值："+str(y_test[t])+"\n预测值："+str(y_pred[0])
12    plt.title(title)                  #设置子图标题
13 plt.show()                          #显示图形
```

图 4 应用顺序网络模型

在测试集中随机选择 5 张图片，使用训练完成的网络模型进行预测。① 使用随机函数生成随机整数赋值给变量 t，t 代表测试集中选中数据的下标；② 将该数据平展为一维张量并存入 x；③ 对张量 x 应用顺序网络模型并得到概率最

大的下标，即预测值。在子图中绘制手写数字图像，并在子图标题上显示标签值和预测值。在子图中绘制手写数字图像，并在子图标题上显示标签值和预测值。结果如图 5。



图 5 识别效果

(2) 实训项目

参考教材 P43-P57，完成使用 `sklearn` 搭建多层神经网络进行红酒数据集分类的实验。